



고려대학교
KOREA UNIVERSITY



AI Security

- Lecture 02. Logistic Regression & SVM

Artificial Intelligence Research (AIR) LAB

<https://air.korea.ac.kr/>

School of Cybersecurity

Korea University

Sangkyun Lee (이상근)

2025. Spring Semester

확률의 개요

- 가능한 모든 사건의 집합 Ω
 - 예: 동전을 두 개 던질 때: $\Omega = \{HH, HT, TH, TT\}$
- 관심 대상 사건의 집합
 - 예: 앞면이 한번 나오는 사건 집합 $A = \{HT, TH\}$
- 확률: 가능한 모든 사건의 집합 대비, 관심 대상 집합의 크기 비율:

$$P(A) = \frac{|A|}{|\Omega|} = \frac{2}{4} = \frac{1}{2}$$

조건부 확률

- 특정 조건 하에서 관심 대상 사건 집합의 비율

- 동전 세개를 던지는 경우

- 전체 사건 집합: $\Omega = \{HHH, HHT, \dots, TTT\}$

- 관심대상 사건 집합 A: 앞면이 나오는 동전이 한개인 경우 $A = \{HTT, THT, TTH\}$

- 관심대상 사건 집합 B: 첫번째 동전이 앞면인 경우

- Ω 중에서 생각하는 경우: $B = \{HHH, HHT, HTH, HTT\}$

$$P(B) = \frac{4}{8}$$

- A 중에서 생각하는 경우: $B = \{HTT\}$

$$P(B|A) = \frac{1}{3}$$

관심 대상 사건 집합을 정의하는 두가지 방법

- 직접 정의

- A = 동전을 세 개 던질 때 앞면인 동전이 1개인 사건의 집합 $A = \{HTT, THT, TTH\}$

- 간접 정의

- $X(\text{사건})$ = 사건을 보고 동전의 개수를 반환하는 함수
 - $A = \{ \text{사건: } X(\text{사건}) = 1 \}$ 로 정의.
 - 더 간단히, $X=1$ 인 사건(집합)이라 표기
 - X 를 확률 변수 (random variable)이라 부름

로지스틱 회귀를 통한 클래스 확률 모델링

1 감독학습: 이진 분류

로지스틱 회귀는 이진 분류 문제에서 각 클래스에 속할 확률을 모델링하는 방법임. 입력 데이터가 주어졌을 때 특정 클래스에 속할 확률을 예측함.

2 오즈비(Odds-ratio)

확률 p 와 $(1-p)$ 의 비율로, 수식으로는 $p/(1-p)$ 로 표현됨. 이는 특정 이벤트가 발생할 확률과 발생하지 않을 확률의 비율을 나타냄.

3 모델링: 로짓 함수(Logit function)

로짓 함수는 오즈비의 자연로그로, $\log(p/(1-p))$ 로 표현되며 $(-\infty, +\infty)$ 범위의 값을 가짐. 이 함수를 선형 함수로 모델링함: $\text{logit}(p) = w^T x = z$

입력데이터 $x \in X$

출력데이터 $y \in \{0, 1\}$

로지스틱 회귀

입력데이터 $x \in X$

출력데이터 $y \in \{0, 1\}$

Log Odds Ratio:
(Logit)

$$\log \frac{p}{1 - p}$$

바이너리 의사 결정에 있어 중요한 수치

로지스틱 회귀의 모델링:

$$\log \frac{p}{1 - p} = w^T x$$

(선형모델)

(로지스틱) 시그모이드 함수

□ 최종 모델링 목표: $\mathbb{P}(Y = 1|X = x)$

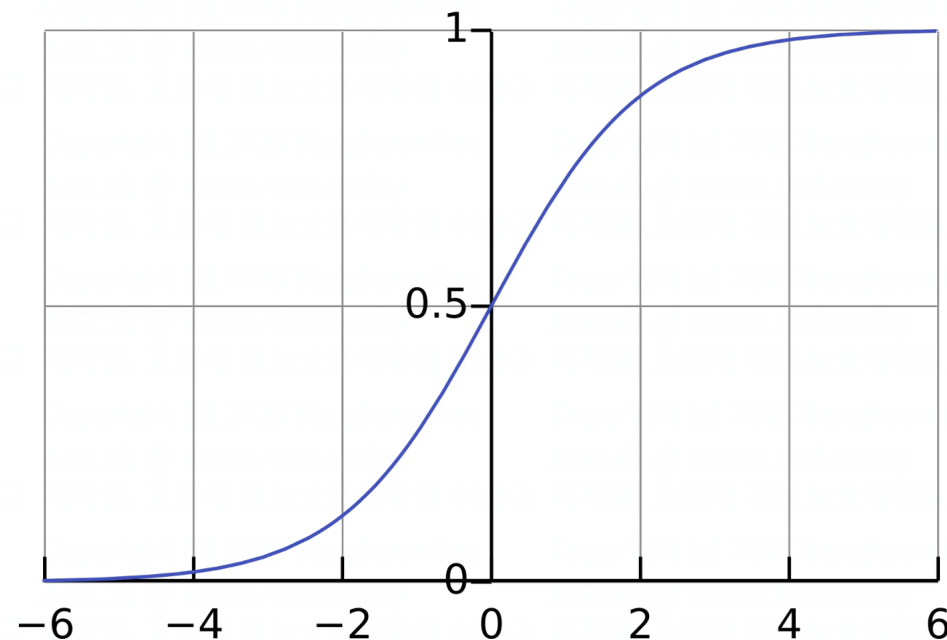
□ 로지스틱 회귀의 모델링:

$$\log \frac{p}{1-p} = w^T x$$

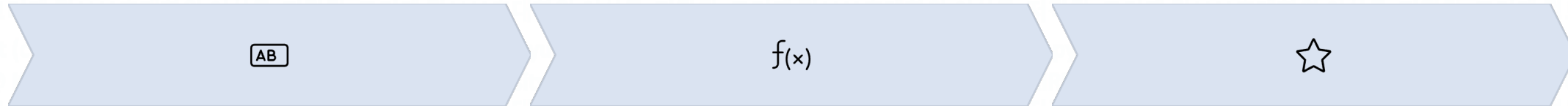
$$p = \mathbb{P}(Y = 1|X = x) = \frac{1}{1 + e^{-w^T x}}$$

□ 로지스틱 회귀의 모델링의 결과:

$$p = \sigma(w^T x)$$



로지스틱 회귀의 직관과 조건부 확률



입력 데이터

특성 벡터 x 가 주어짐. 이 데이터는 다차원 공간의 한 점으로 표현될 수 있음.

선형 함수

가중치 벡터 w 와 입력 x 의 내적을 계산: $z = w^T x$. 이는 데이터를 선형적으로 변환하는 과정임.

확률 변환

시그모이드 함수를 통해 선형 출력을 0과 1 사이의 확률로 변환: $p = \sigma(z)$. 이 값은 입력 x 가 클래스 1에 속할 조건부 확률을 나타냄.

로지스틱 회귀는 조건부 확률 $P(Y=1|X=x)$ 를 모델링함. 이는 입력 x 가 주어졌을 때 출력 Y 가 1일 확률을 의미함. 이 모델은 이진 분류 문제에서 널리 사용되며, 다중 클래스 분류로도 확장 가능함.

로지스틱 회귀의 다른 관점 = 퍼셉트론

Inputs

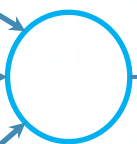
x_1

x_2

$\vdots$

x_n

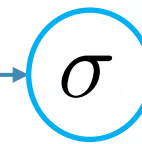
Logit



$$\sum_{i=1}^n w_i x_i = w^T x$$

선형모델

Sigmoid

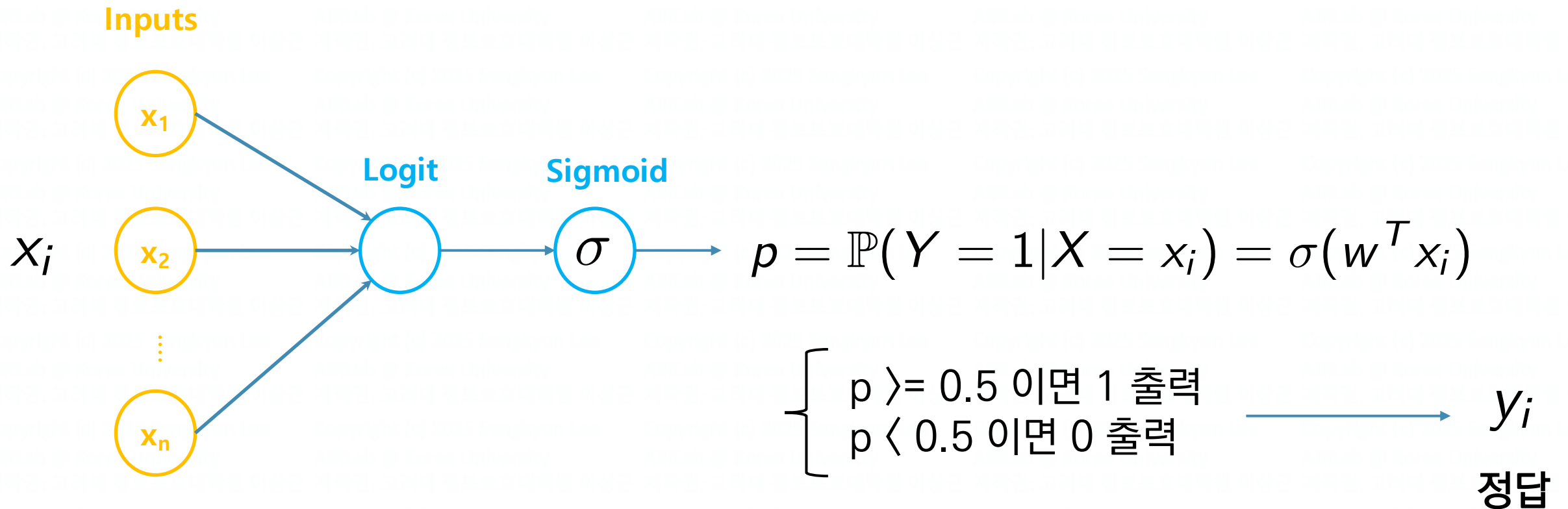


$$\sigma(w^T x) = p = \mathbb{P}(Y = 1 | X = x)$$

학습 문제

학습 데이터 $\{(x_i, y_i)\}_{i=1}^n : x_i \in X, y_i \in \{0, 1\}$

에 대해, 정답을 잘 맞추는 학습 파라미터 w 를 어떻게 찾을 것인가?



최대 우도 추정 (MLE, Maximum Likelihood Estimation)

- 초등학교에서 배운 주사위 던지기 문제를 떠올려 보자:
 - (모델링)모든 눈이 같은 확률로 나오는 공평한 주사위: $P(X = i) = 1/6, i = 1, 2, \dots, 6$
 - 주사위를 10번 던짐 (각 시행은 서로 독립)
 - 다음 사건(데이터)을 관측할 확률은? $\{3, 6, 1, 4, 2, 2, 4, 5, 6, 3\}$

- 반대로, 다음의 데이터가 주어짐:

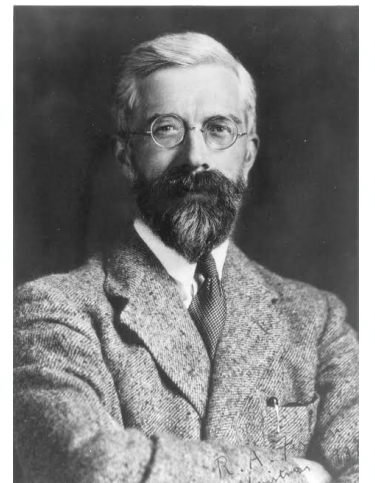
$\{3, 6, 1, 4, 2, 2, 4, 5, 6, 3\}$

- 각 눈이 나올 확률 $P(X = i)$ 의 좋은 추정치는?

R.A. Fisher (1890~1962)



at 23



MLE

- 사건의 관측치 (데이터): $\{o_1, o_2, \dots, o_n\}$ $o_i \stackrel{i.i.d.}{\sim} \mathbb{P}(O; \theta)$
- Likelihood 함수 $L(\theta) = \mathbb{P}(o_1, o_2, \dots, o_n; \theta)$
 - 모델 (파라미터: θ)를 가정했을 때, 이 특정 데이터를 관측할 확률
 - 모델 파라미터 θ 의 함수
- MLE: likelihood를 최대화하는 모델 파라미터 θ 를 찾는 절차

$$\max_{\theta} L(\theta)$$

$$\min_{\theta} -LL(\theta) = -\log L(\theta)$$

Negative log likelihood (NLL)

MLE for Binary Classification

로지스틱 회귀 (라벨의 베르누이 분포)

$$P(Y = 1|X = x; w) = \sigma(f(w, x_i)) = \frac{1}{1 + \exp(-f(w, x_i))}$$

$$P(Y = 0|X = x; w) = 1 - \sigma(f(w, x_i))$$

(Conditional) Log likelihood 함수:

$$\begin{aligned}\log P(y_1, \dots, y_n | x_1, \dots, x_n; w) &= \log \prod_{i=1}^n P(y_i | x_i; w) \\ &= \log \prod_{i=1}^n P(y_i = 1 | x_i; w)^{y_i} P(y_i = 0 | x_i; w)^{1-y_i} \\ &= \sum_{i=1}^n \{y_i \log \sigma(f(w, x_i)) + (1 - y_i) \log(1 - \sigma(f(w, x_i)))\}\end{aligned}$$

학습 (Training)

학습 데이터에 대해 NLL 함수를 최소화하는 학습 파라미터 w 를 찾는 과정

$$\min_{w \in \mathbb{R}^p} -\log P(y_1, \dots, y_n | x_1, \dots, x_n; w)$$

$$\min_{w \in \mathbb{R}^p} -\sum_{i=1}^n \{y_i \log f(w, x_i) + (1 - y_i) \log(1 - f(w, x_i))\}$$

일반적으로, 모든 기계학습/딥러닝 방법론들은 다음과 같은 형태의 학습 문제를 풀게 됨

$$\min_{w \in \mathbb{R}^p} \sum_{i=1}^n \ell(w; x_i, y_i)$$

 Loss function

Training Problem: A General Form

$$\min_{w \in \mathbb{R}^p} \sum_{i=1}^n \ell(w; x_i, y_i)$$

Task	Labels	Loss function
Regression	$y_i \in \mathbb{R}$	$\ell(w; x_i, y_i) = (y_i - f(w, x_i))^2$
Classification (Binary)	$y_i \in \{0, 1\}$	$\ell(w; x_i, y_i) = y_i \log f(w, x_i) + (1 - y_i) \log(1 - f(w, x_i))$
Classification (Multi-Class)	$y_i \in \{1, \dots, K\}$	$\ell(w; x_i, y_i) = \sum_{k=1}^K I_{y_i=k} \log \text{softmax}(f(w, x_i))_k$

수치 최적화

최적화 변수 $\min_{w \in \mathbb{R}^p}$ 목적 함수 $f(w)$

subject to $\boxed{\begin{array}{l} g(w) \leq 0 \\ h(w) = 0 \end{array}}$ 제약조건

- 목적함수, 제약조건에 따라 Linear program (LP), quadratic program (QP), integer program, mixed-integer program, quadratically-constrained QP (QCQP), second-order cone program (SOCP), semi-definite program (SDP) 등 형태를 정의
- 해당 형태를 빨리 풀어낼 수 있는 상용 / 오픈 solver 존재

스토케스틱 경사하강법 (SGD, Stochastic Gradient Descent)

$$\vec{w}^* \in \arg \min_{\vec{w} \in \mathbb{R}^n} J(\vec{w}) = \frac{1}{m} \sum_{i=1}^m \ell(y_i, f(\vec{w}, \vec{x}_i))$$

- Initialize w randomly
- For N epochs
 - For a random training example $J_i(w) = \ell(y_i, f(\vec{w}, \vec{x}_i))$
 - Compute stochastic (sub)gradient of loss: $\frac{\partial J_i(w)}{\partial w}$
 - Update w :
$$w = w - \eta \frac{\partial J_i(w)}{\partial w}$$

Learning rate



Mini-Batch SGD

Use a small subset of examples per update, to reduce variance of gradient estimates

- Initialize w randomly
- For N epochs

- For minibatch samples $J_i(w) = \ell(y_i, f(\vec{w}, \vec{x}_i))$

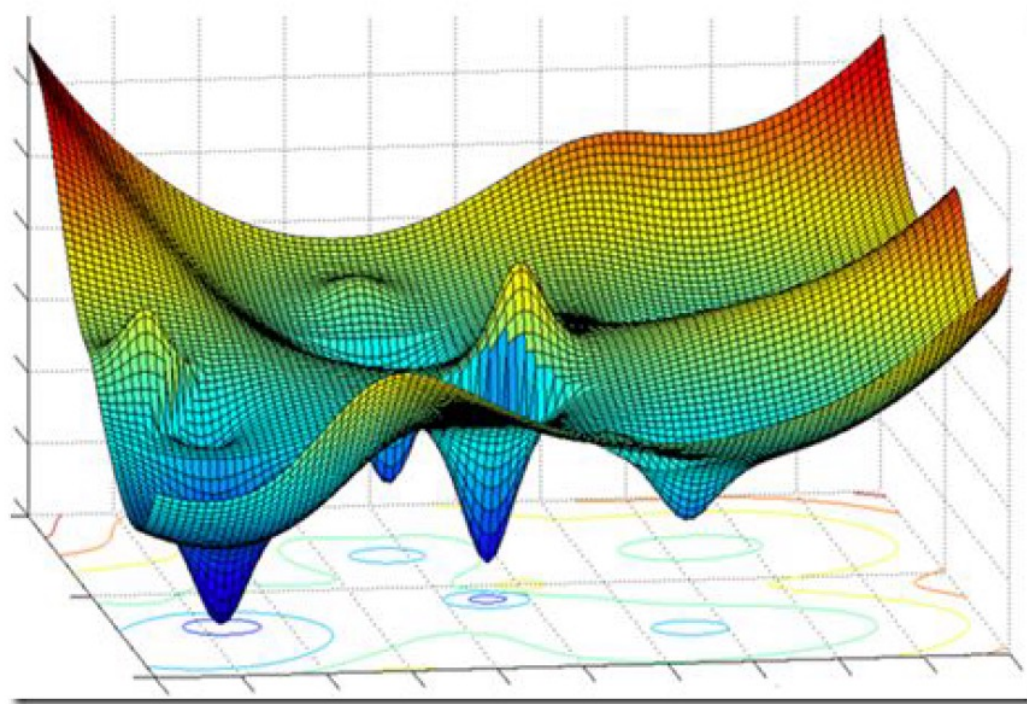
- Compute stochastic (sub)gradient of loss: $\frac{\partial J(w)}{\partial w} \approx \frac{1}{B} \sum_i^B \frac{\partial J_i(w)}{\partial w}$

- Update w :

$$w = w - \eta \frac{\partial J(w)}{\partial w}$$

DNN Objective Functions

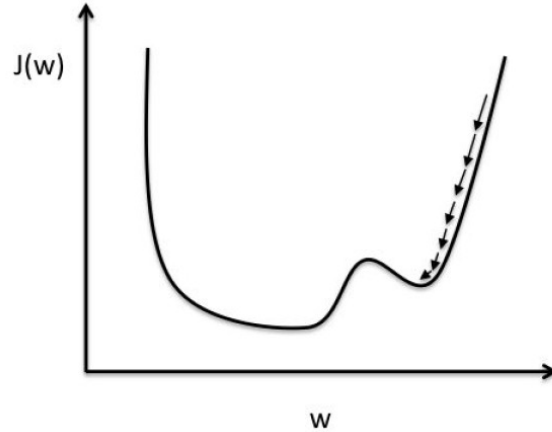
Objective functions of DNN are typically nonconvex, with lots of local minimizers and saddle points



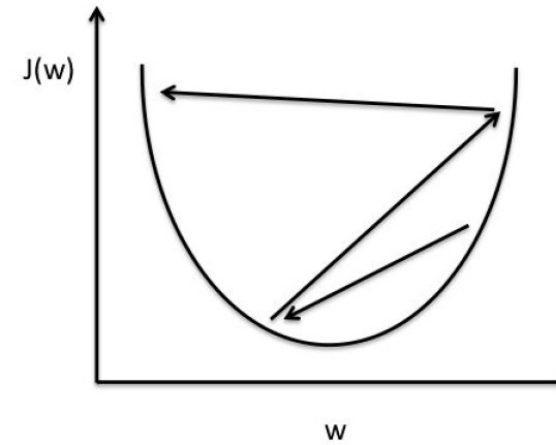
Learning Rate is Important

$$w = w - \eta \frac{\partial J(w)}{\partial w}$$

How to choose the learning rate?



Small learning rate: Many iterations until convergence and trapping in local minima.



Large learning rate: Overshooting.

Adaptive Learning Rate

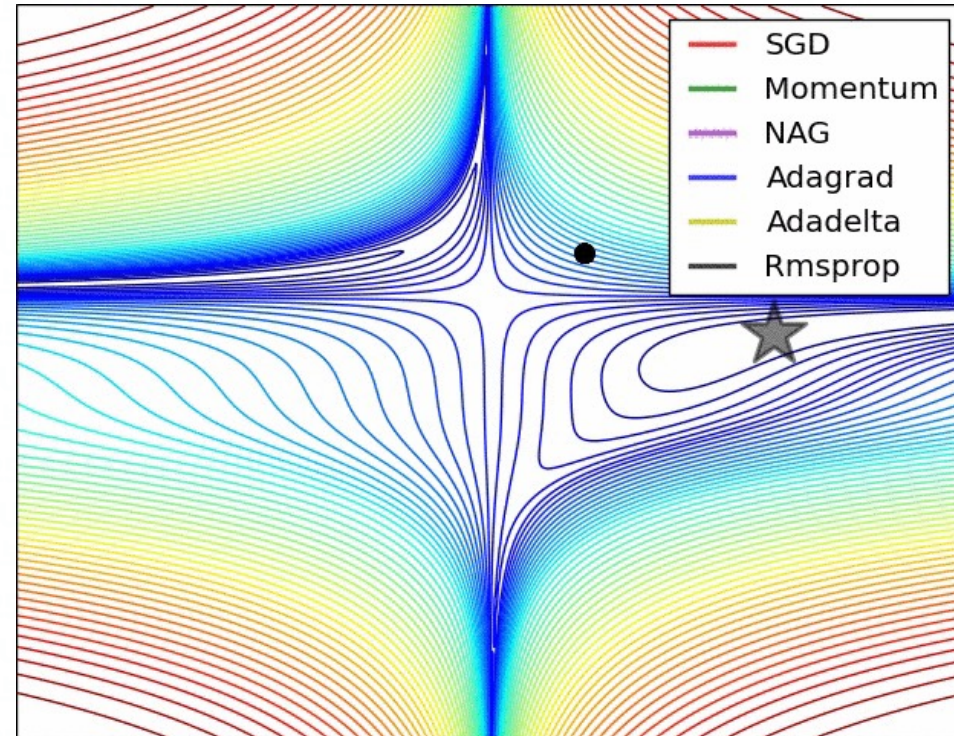
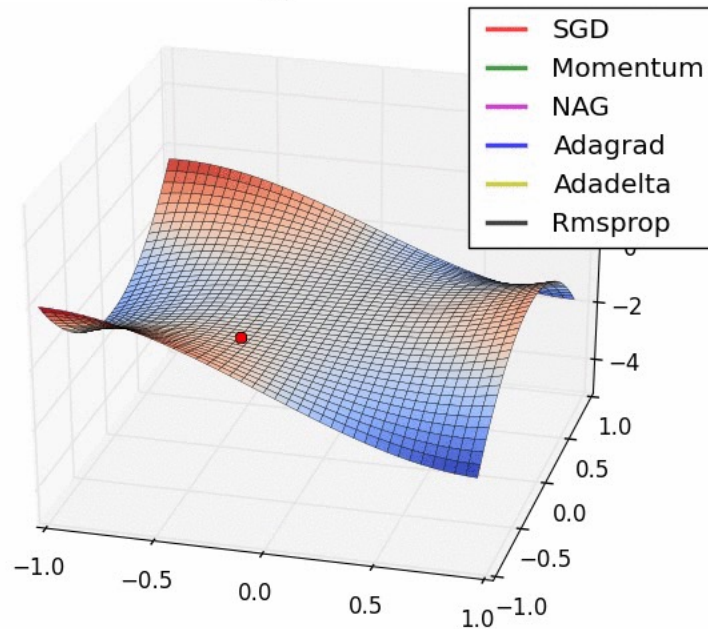
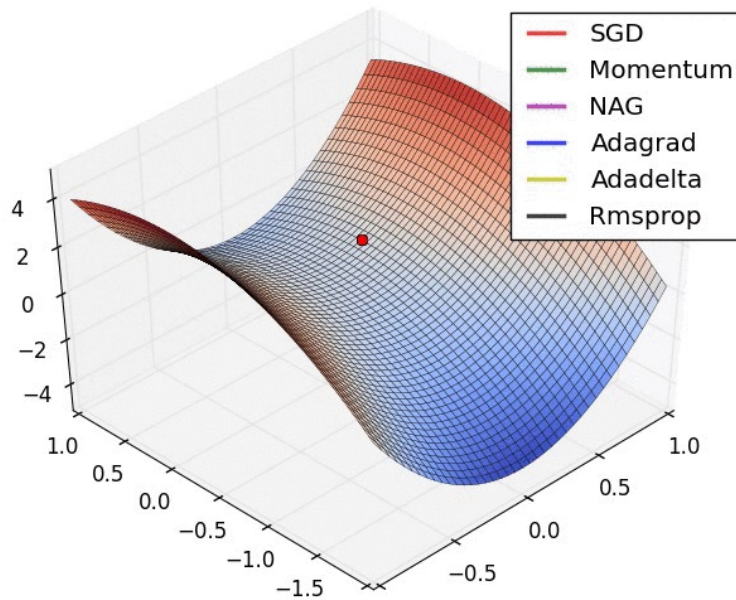
Learning rates can be chosen adaptively to:

- How large the gradient is
- How fast learning is happening
- Magnitude of particular weights
-

Adaptive learning rate algorithms:

ADAM, Momentum, NAG, Adagrad, Adadelata, RMSProp, ...

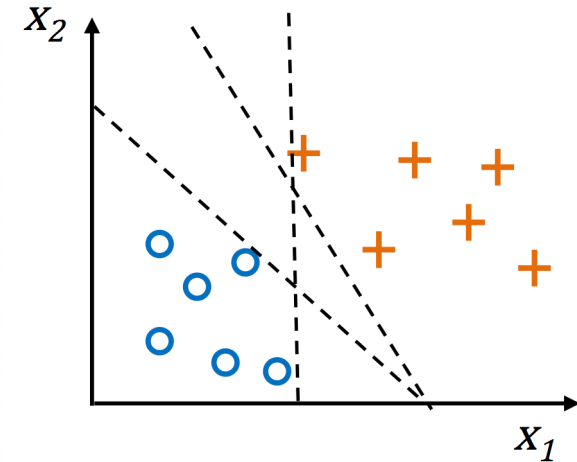
Adaptive Learning Rate Algorithms



SVM

SVM (Support Vector Machine)

- V. Vapnik 교수의 방법론
 - 통칭, 기계학습의 아버지
 - Cortes, Corinna; Vapnik, Vladimir N., Support-vector networks, Machine Learning, 20 (3): 273-297 (1995)
 - The nature of statistical Learning Theory, Springer, 1999
- 아이디어: 두 포인트 클라우드 사이의 “마진”을 고려
 - 예측 오차를 최소화하는 동시에
 - ”마진”을 최대화
- SVM이 흥미로운 이유
 - 오버피팅이 드물게 발생
 - “커널”을 이용, 비선형 결정경계를 갖도록 기능 확장 가능



SVM

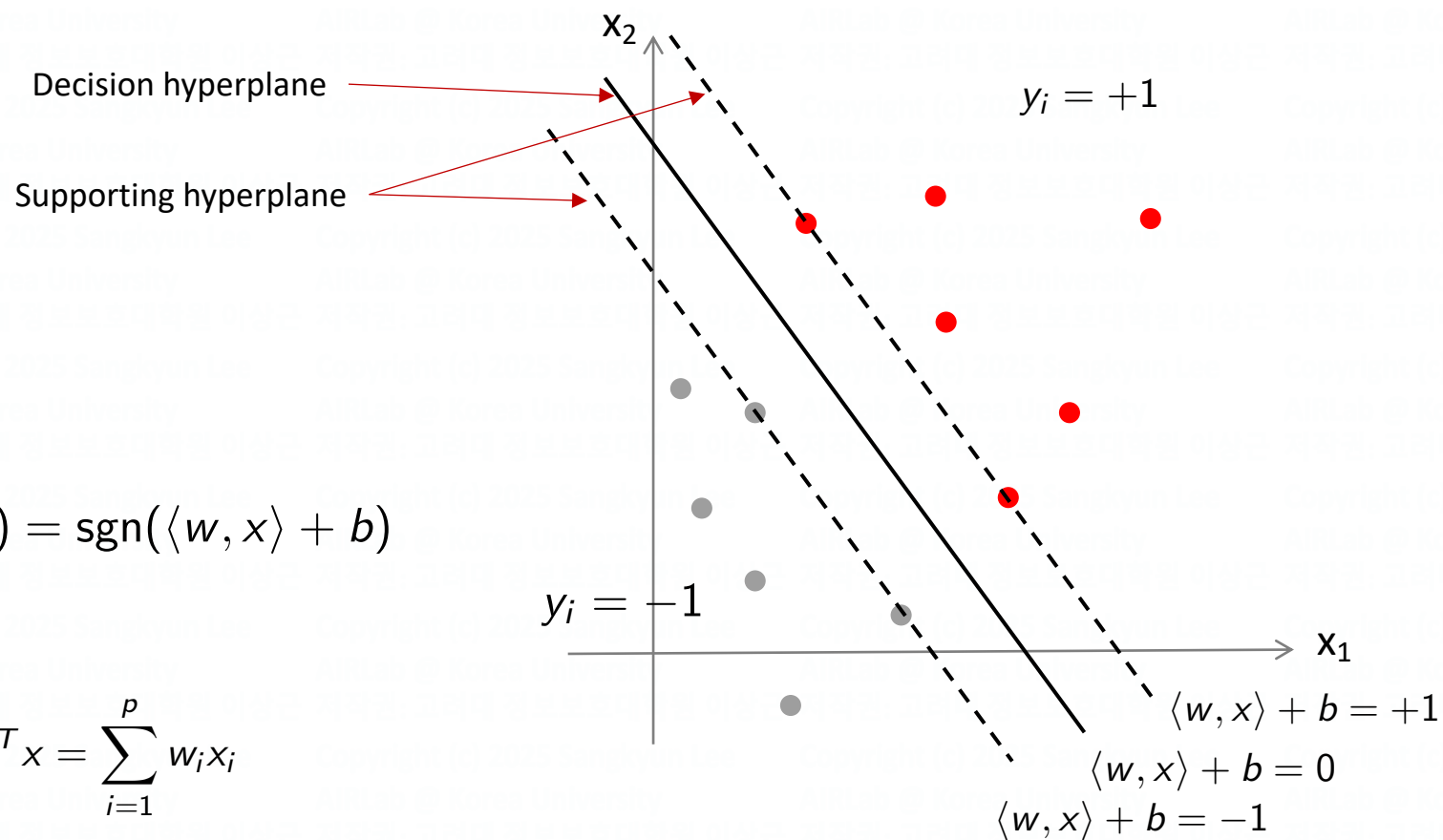
- Data

- Input: $x \in \mathbb{R}^p$ ($x \in X$ in general)
- Label: $y \in \{-1, +1\}$

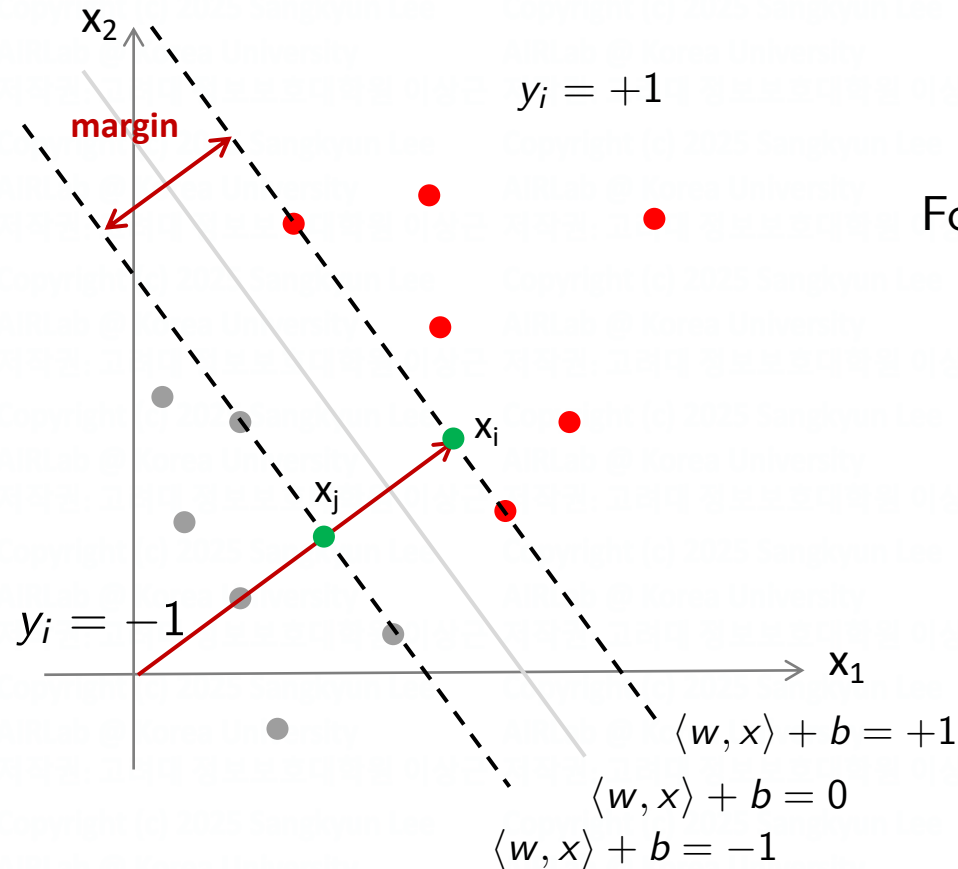
- Decision

$$f_w(x) = \text{sgn}(\langle w, x \rangle + b)$$

$$\langle w, x \rangle = w^T x = \sum_{i=1}^p w_i x_i$$



Margin of SVM



$$\langle w, x_i \rangle + b = +1$$

$$\langle w, x_j \rangle + b = -1$$

$$\Rightarrow \langle w, x_i - x_j \rangle = 2$$

$$\text{For } x_i - x_j = t \frac{w}{\|w\|_2}, t \in \mathbb{R},$$

$$t = \frac{2}{\|w\|_2}$$

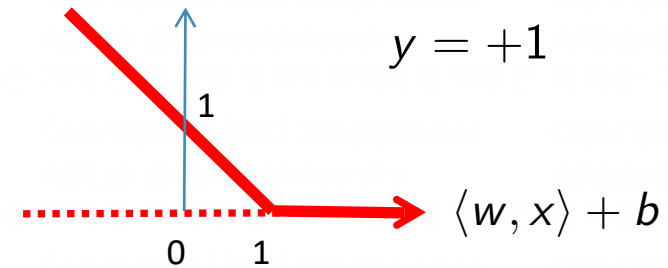
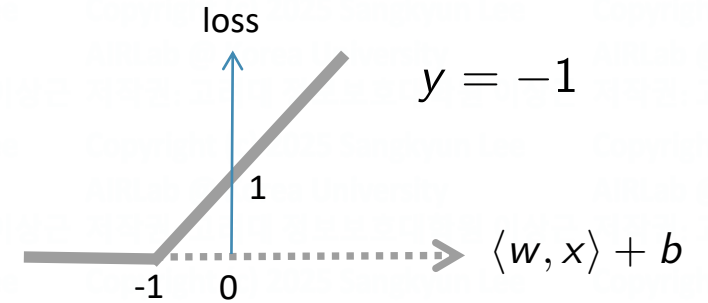
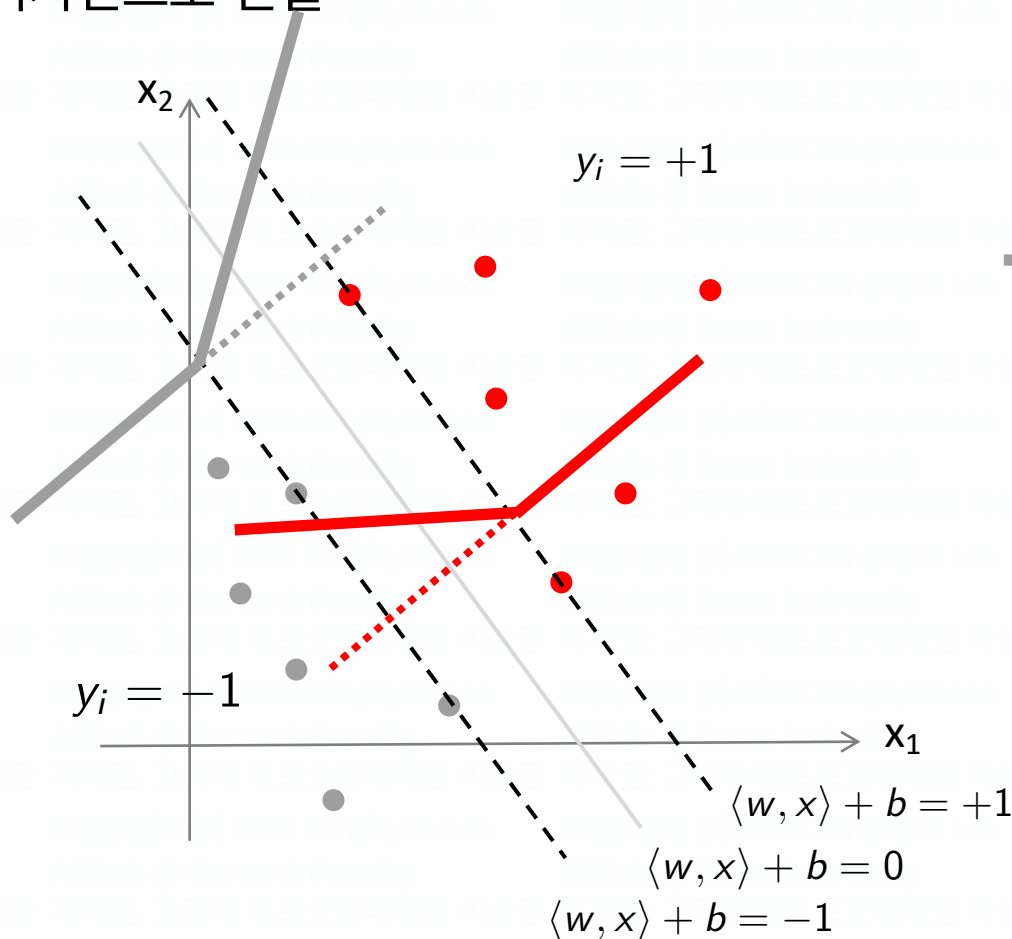
Learning Goal 1 (max. margin)

$$\min_w \frac{1}{2} \|w\|_2^2$$

Learning Goal 2 (Min. Prediction Error)

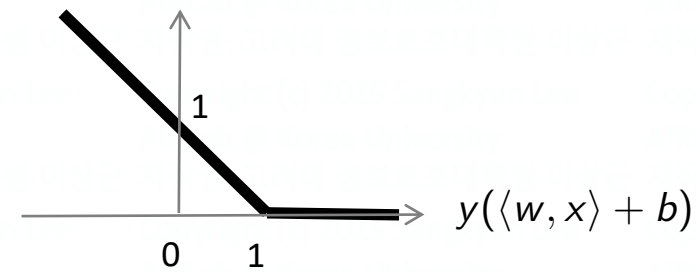
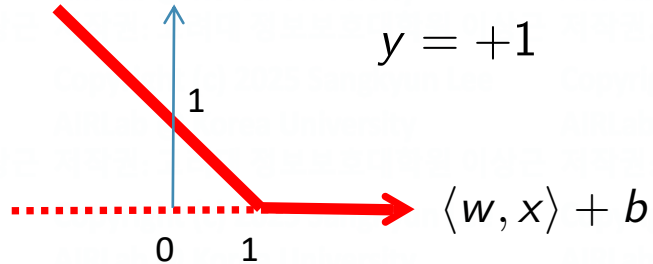
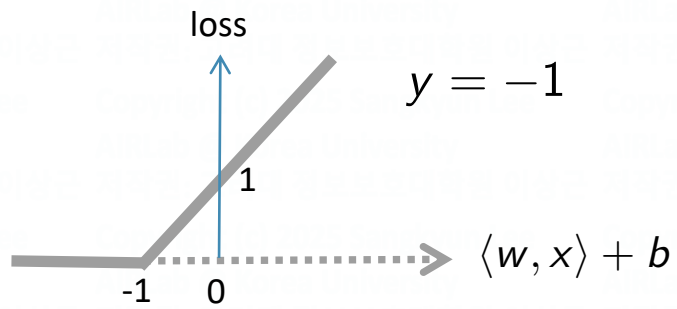
- 마진 영역에 데이터 포인트가 놓이는 상황을 최대한 배제

→ 손실 함수의 디자인으로 연결



Learning Goal 2 (Min. Prediction Error)

- Hinge loss function



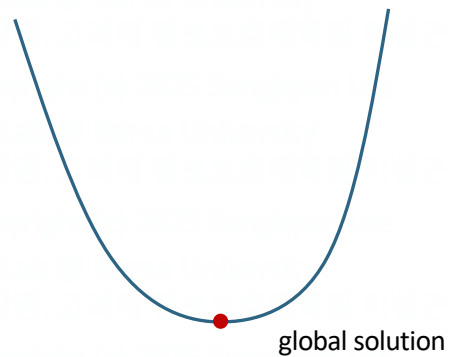
$$\max\{1 - y(\langle w, x \rangle + b), 0\}$$

Learning Goal 2 (min. pred error)

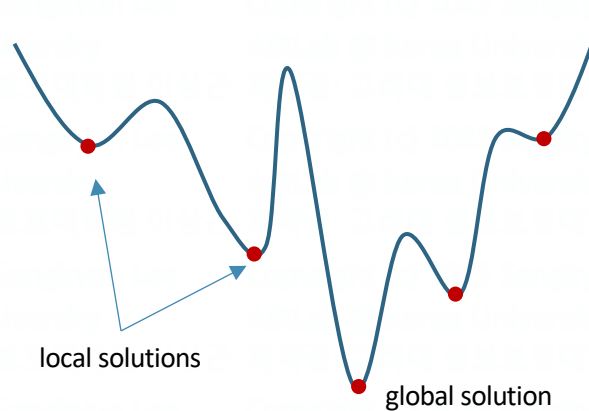
$$\min_{w, b} \sum_{i=1}^n \max\{1 - y_i(\langle w, x_i \rangle + b), 0\}$$

Convex vs. Non-convex Optimization

- ML methods
 - Based on convex optimization
 - Logistic regression, SVM, ...
 - Based on non-convex optimization
 - Deep neural nets with nonlinear activations (DNN, CNN, RNN, ...)
 - NMF
- Shape of loss functions



Convex



Non-convex

C-SVM

$$\min_{w \in \mathbb{R}^p, b \in \mathbb{R}, \xi \in \mathbb{R}^n} \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \xi_i$$

subject to $\xi_i \geq 1 - y_i(\langle w, x_i \rangle, b), i = 1, \dots, n$

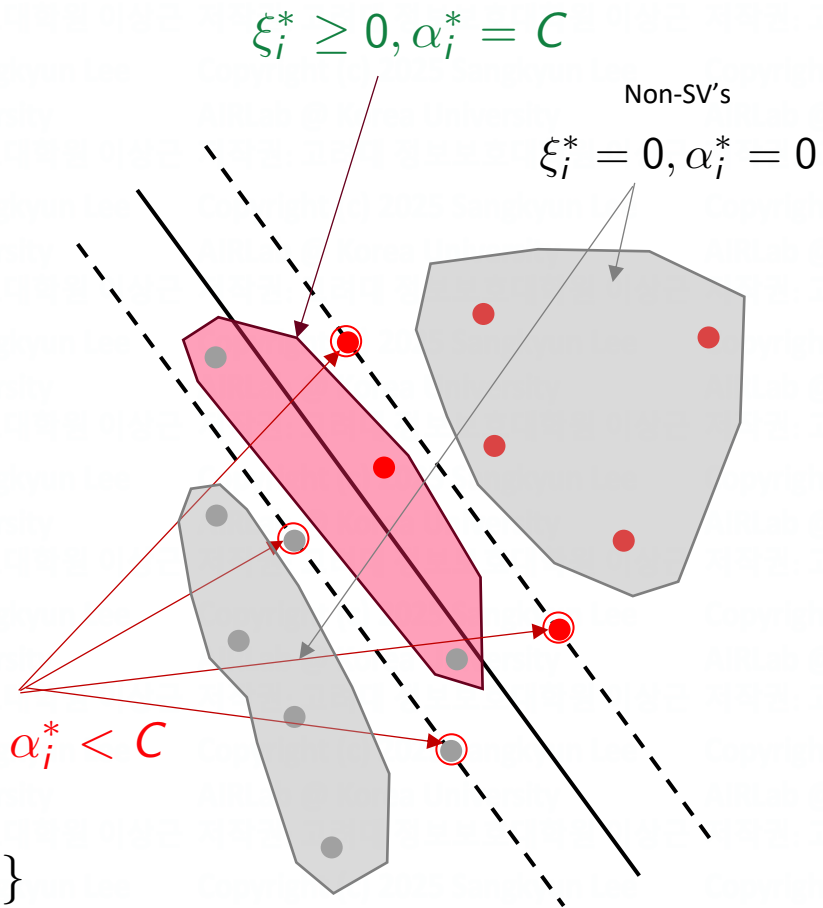
$\xi_i \geq 0, i = 1, \dots, n$

KKT (Karush-Kuhn-Tucker) conditions:
the conditions satisfied by the
solution

Insights from the KKT conditions:

$$w^* = \sum_{i=1}^n \alpha_i^* x_i \quad \begin{array}{l} \text{dual variable} \\ 0 \leq \alpha_i^* \leq C \end{array}$$

Support vector set: $\{i \in \{1, \dots, n\} : \alpha_i^* > 0\}$

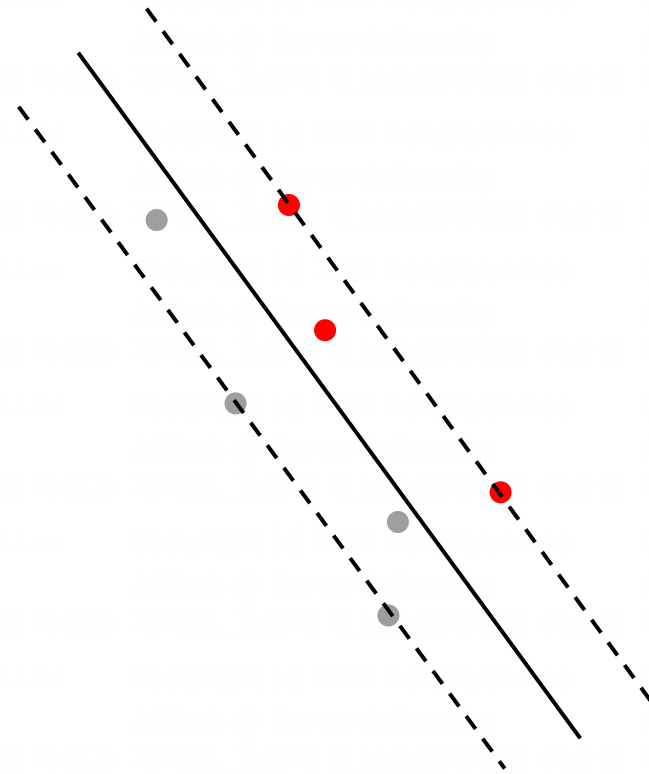
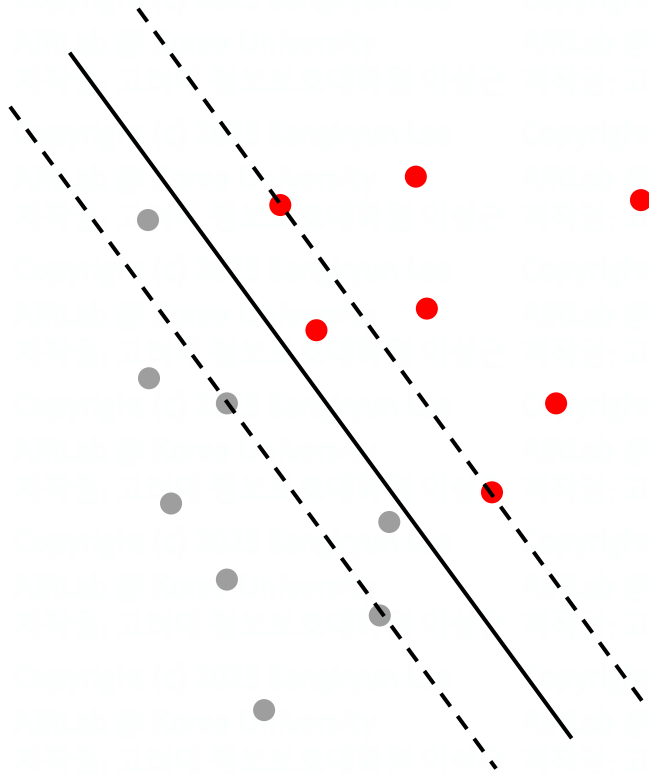


C-SVM

$$\begin{aligned} \min_{w \in \mathbb{R}^p, b \in \mathbb{R}, \xi \in \mathbb{R}^n} \quad & \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \xi_i \\ \text{subject to} \quad & \xi_i \geq 1 - y_i(\langle w, x_i \rangle, b), \quad i = 1, \dots, n \\ & \xi_i \geq 0, \quad i = 1, \dots, n \end{aligned}$$

$$w^* = \sum_{i \in SV} \alpha_i^* x_i$$

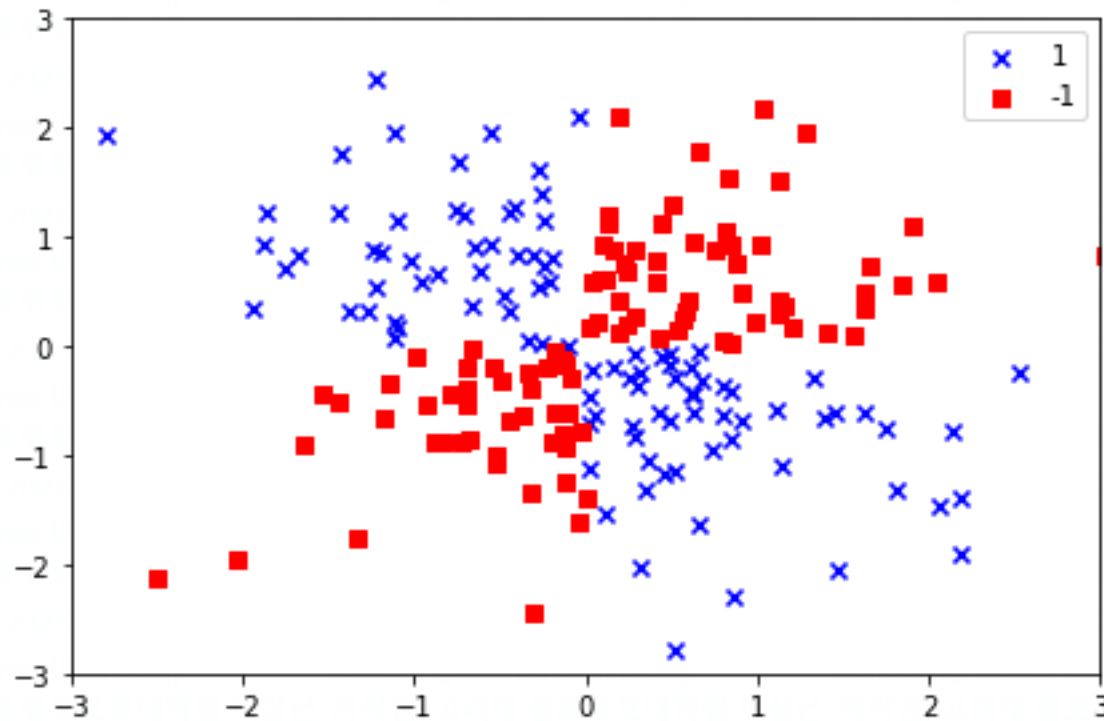
b^* can be computed using only SV's



The same classifier will be obtained!

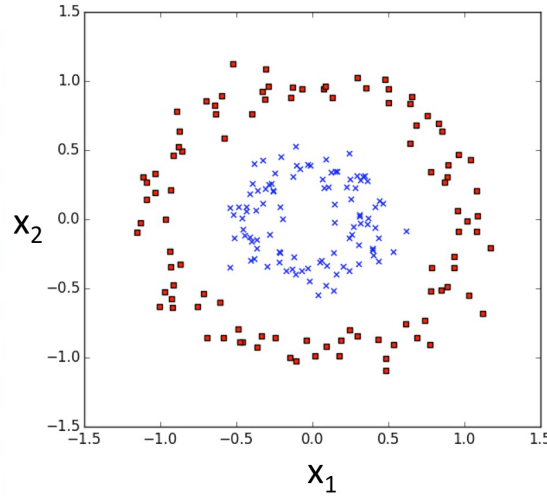
비선형 분류 문제

- Data with XOR class pattern

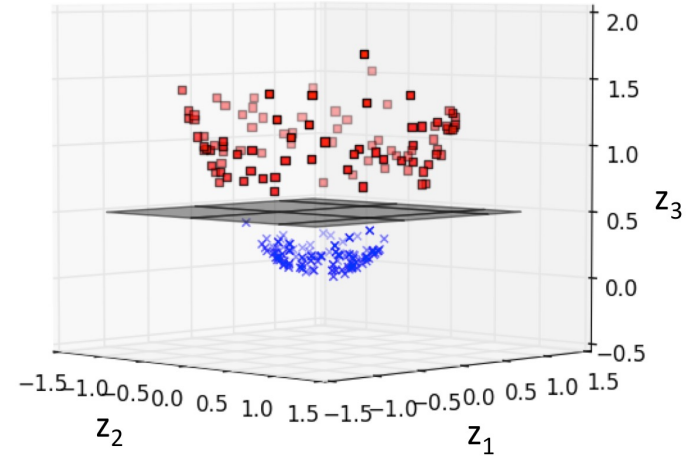


Feature Map

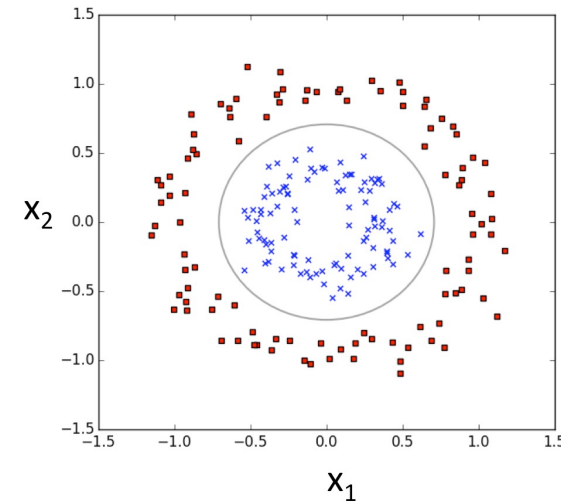
$$\phi(x_1, x_2) = (z_1, z_2, z_3) = (x_1, x_2, x_1^2 + x_2^2)$$



ϕ



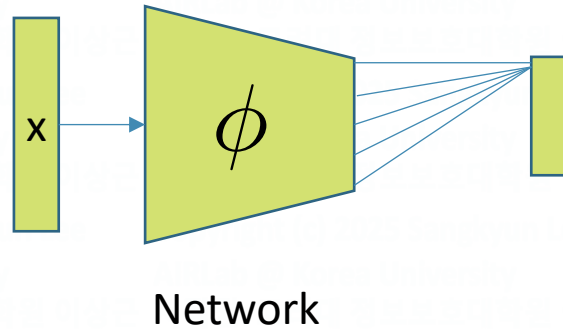
ϕ^{-1}



데이터의 표현형 (representation)을 바꾸는 것이 핵심!

Representation Learning

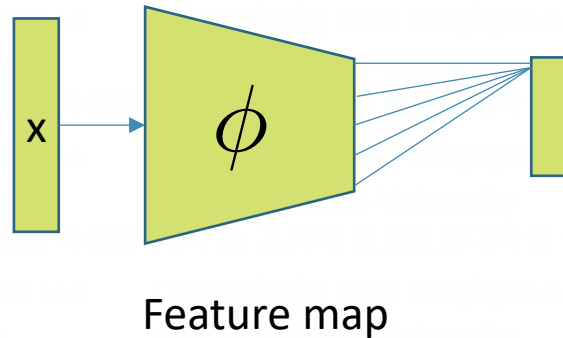
Deep neural nets:



$$\sigma\left(\sum_i w_i h_i + b\right)$$

Decision is a linear function of the final hidden outputs

SVM:



$$\text{sgn}(\langle w_i, \phi(x) \rangle + b)$$

Decision is a linear function of the transformed input

커널 트릭

- 피쳐 변환을 직접 설계하는 것은 어려움
- 대신, 커널 함수를 정의

$$k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

$$K \in \mathbb{R}^{n \times n}$$

- 다음의 커널은 유효한 피쳐 변환을 정의함이 알려져 있음

- Linear kernel

$$k(x_i, x_j) = \langle x_i, x_j \rangle$$

- Polynomial kernel

$$k(x_i, x_j) = (\langle x_i, x_j \rangle + 1)^d$$

- Gaussian (RBF: radial basis function) kernel

$$k(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|_2^2)$$

- Sigmoid kernel

$$k(x_i, x_j) = \tanh(b \langle x_i, x_j \rangle + a)$$

▼ Nonlinear Classification

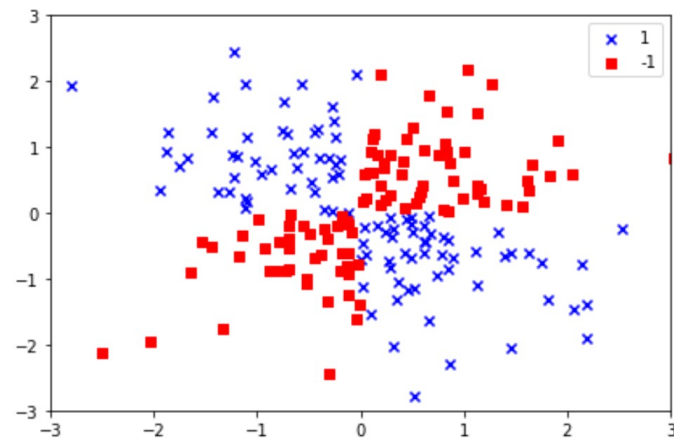
▼ XOR Toy Data

```
[ ] import matplotlib.pyplot as plt
import numpy as np

np.random.seed(1)
X_xor = np.random.randn(200, 2)
y_xor = np.logical_xor(X_xor[:, 0] > 0,
                       X_xor[:, 1] > 0)
y_xor = np.where(y_xor, 1, -1)

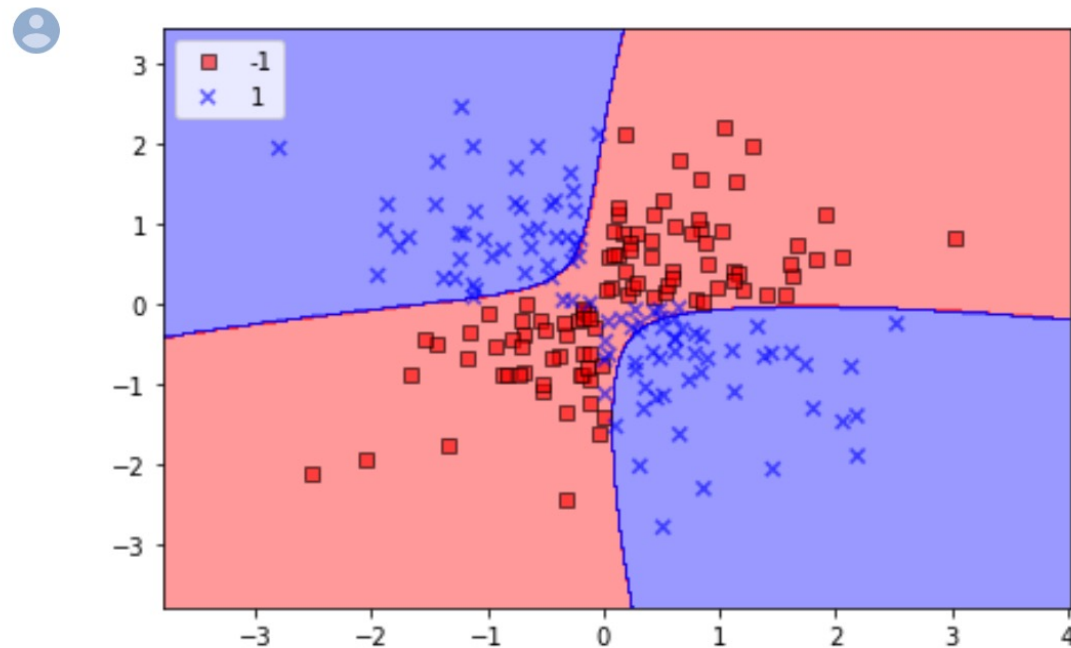
plt.scatter(X_xor[y_xor == 1, 0],
            X_xor[y_xor == 1, 1],
            c='b', marker='x',
            label='1')
plt.scatter(X_xor[y_xor == -1, 0],
            X_xor[y_xor == -1, 1],
            c='r',
            marker='s',
            label='-1')

plt.xlim([-3, 3])
plt.ylim([-3, 3])
plt.legend(loc='best')
plt.tight_layout()
plt.show()
```



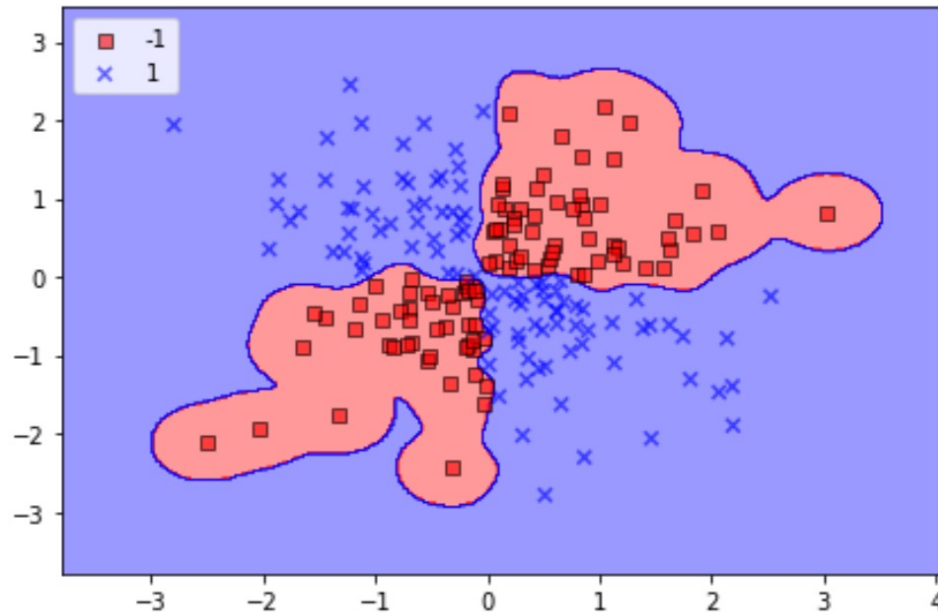
▼ SVM on XOR data

```
#  
# SVM on XOR data  
#  
svm = SVC(kernel='rbf', random_state=1, gamma=.1, C=10)  
svm.fit(X_xor, y_xor)  
plot_decision_regions(X_xor, y_xor,  
                      classifier=svm)  
  
plt.legend(loc='upper left')  
plt.tight_layout()  
plt.show()
```



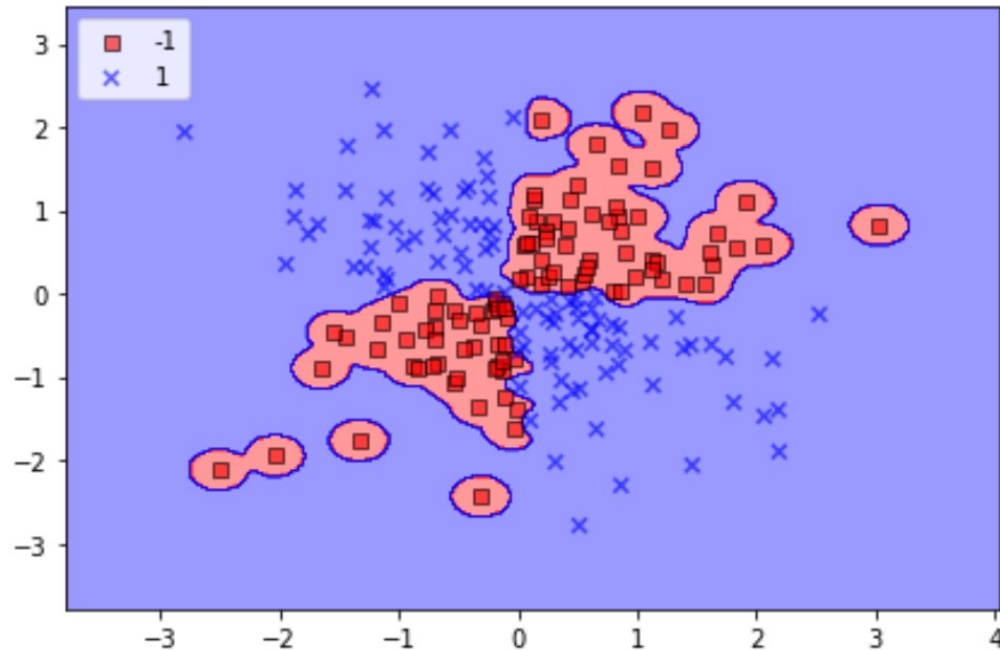
▼ SVM on XOR data

```
[9] #  
# SVM on XOR data  
#  
svm = SVC(kernel='rbf', random_state=1, gamma=10, C=10)  
svm.fit(X_xor, y_xor)  
plot_decision_regions(X_xor, y_xor,  
                      classifier=svm)  
  
plt.legend(loc='upper left')  
plt.tight_layout()  
plt.show()
```



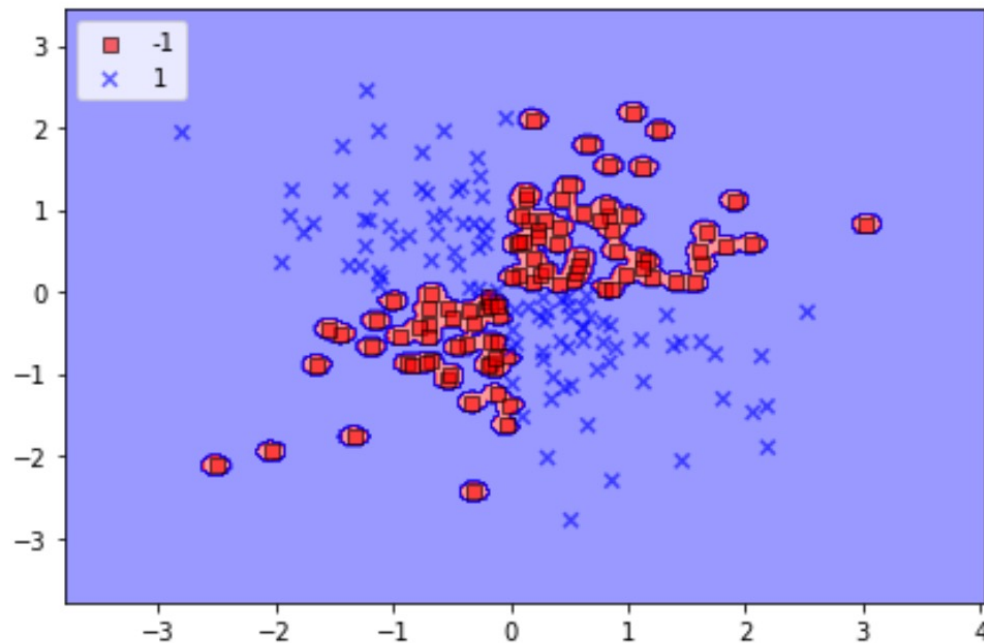
▼ SVM on XOR data

```
[10] #  
# SVM on XOR data  
#  
svm = SVC(kernel='rbf', random_state=1, gamma=50, C=10)  
svm.fit(X_xor, y_xor)  
plot_decision_regions(X_xor, y_xor,  
                      classifier=svm)  
  
plt.legend(loc='upper left')  
plt.tight_layout()  
plt.show()
```



▼ SVM on XOR data

```
[13] #  
# SVM on XOR data  
#  
svm = SVC(kernel='rbf', random_state=1, gamma=250, C=10)  
svm.fit(X_xor, y_xor)  
plot_decision_regions(X_xor, y_xor,  
                      classifier=svm)  
  
plt.legend(loc='upper left')  
plt.tight_layout()  
plt.show()
```



Thank You